

Beyond Single view Decoding: Dual-view Map Inference from Trajectories via Primal-Dual Graphs Co-generation

Wenyu Wu
East China Normal University
Shanghai, China
52275903019@stu.ecnu.edu.cn

Jiafan Liu
East China Normal University
Shanghai, China
51275903116@stu.ecnu.edu.cn

Jiali Mao*
East China Normal University
Shanghai, China
jlmiao@dase.ecnu.edu.cn

Abstract

Digital maps are crucial for web-based location services. The continuous collection of vehicle trajectories has made trajectory data a vital source for map inference. State-of-the-art (SOTA) methods formulate trajectory-based map inference as an image processing task: they first rasterize trajectories into density-based images, then extract keypoints and infer their connections to construct the road map. Despite performing well on standard road structures, these methods still suffer from low topological accuracy in complex scenarios because i) The discrete rasterized representation struggles to capture road adjacency in multi-level road structures. ii) The limited contextual awareness of keypoint-based inferring strategy leads to connectivity misjudgment in dense road areas. To solve these limitations, we propose D²Map, a Dual-view Map Inference Framework via Primal-Dual Graphs Co-generation. To precisely encode road adjacency, we introduce the serialized trajectory view as a complement to the rasterized view to reflect traversable relationships between roads, and devise a strategy-adaptive fusion module that dynamically selects and executes the optimal fusion operator to integrate dual view representations, yielding map element embeddings. To eliminate connectivity errors, we extend road map modeling from a keypoint-centric primal graph to primal and dual graphs. In the dual graph, roads are explicitly modeled as nodes, enabling context-aware topology inference. A co-generation strategy is then employed to jointly infer both graphs while maintaining their geometric consistency. Extensive experiments on two real-world datasets demonstrate the superiority of D²Map, which outperforms SOTA baselines by 11.44% in the TOPO metric.

CCS Concepts

• Computing methodologies → Artificial intelligence.

Keywords

Map inference, Trajectory, Dual-view fusion, Primal-Dual graphs

ACM Reference Format:

Wenyu Wu, Jiafan Liu, and Jiali Mao. 2026. Beyond Single view Decoding: Dual-view Map Inference from Trajectories via Primal-Dual Graphs Co-generation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*,

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792096>

April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792096>

1 Introduction

Digital maps are fundamental to web-based location services like real-time navigation [13, 25], ride-hailing [39, 50], and last-mile delivery [28, 38], which underpin modern urban mobility ecosystems. The continuous collection of vehicle trajectories, with its extensive coverage and low acquisition cost, has established map inference from such data as a key research focus [6, 29, 40].

The evolution of trajectory-based map inference has progressed from heuristic-based methods [3, 5, 15, 18, 29, 33, 37] to learning-based approaches [11, 27, 30, 42]. Early methods adopt heuristic techniques, such as spatial clustering [18, 33] and kernel density estimation [37], to construct the road map. Their performance heavily relies on predefined distance thresholds, for example, the bandwidth in KDE [3], often leading to suboptimal accuracy [30]. To improve inference accuracy, recent methods introduce deep visual learning, casting the task as an image processing one: they first rasterize trajectories into density-based images, encode these inputs into pixel-level embeddings using a deep vision model, and then decode the embeddings into the road map. Initial attempts [11, 27] treat the decoding as a semantic segmentation task, identifying road pixels followed by heuristic post-processing to generate the road map. Such a two-stage pipeline is prone to error accumulation [24]. To solve this limitation, keypoint-based methods [30, 42] employ an end-to-end framework to directly identify keypoints (e.g., points uniformly sampled from roads or road intersections) and infer their relationships, achieving high accuracy in regular scenarios [30]. Nevertheless, these methods still suffer from low accuracy in inferring road topology (i.e., the connectivity relationships between roads) under complex road structures. This shortfall arises from two critical limitations:

Limitation 1: Raster-only representation struggles to capture road adjacency relationship in multi-level road structures. Rasterization-based methods convert trajectories into 2D discretized, fine-grained grids, where each grid corresponds to a pixel in the image. This representation inherently fails to preserve the topological adjacency between roads. As shown in Figure 1 (a), in multi-level road scenarios (e.g., overpasses), roads at different vertical layers (r_0, r_1, r_2) are topologically disconnected, yet their 2D projections intersect. Consequently, existing methods tend to misinterpret these regions as road intersections (highlighted by red rectangles in Figure 1 (b)). To mitigate this issue, a natural solution is to leverage the serialized representation of trajectories. As recorded vehicle travel paths, trajectory point sequences inherently

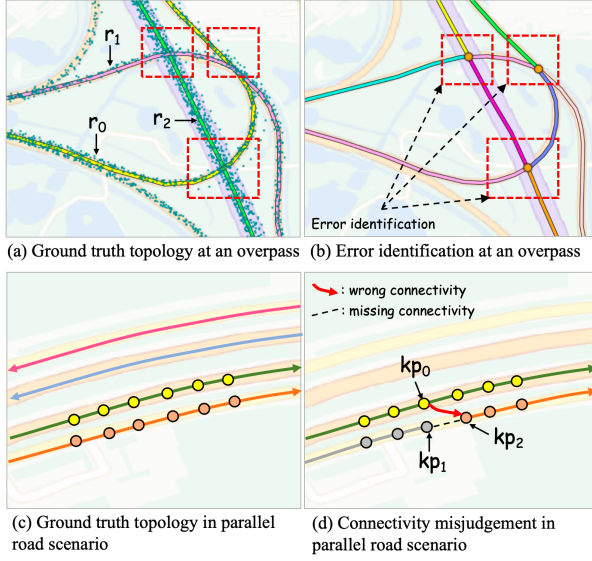


Figure 1: Illustration of low topological accuracy in map inference, where green dots denote the trajectory points, curves and arrows in different colors denote the roads, and circles filled by different colors denote the keypoints

imply traversable relationships between roads, providing a complementary perspective to the rasterized view for road adjacency encoding. However, effectively integrating the two views is non-trivial. First, serialized and rasterized expressions are structurally heterogeneous: the former is an ordered point sequence, while the latter is a discretized, density-based image. Moreover, naive fusion strategies such as feature concatenation [34] or attention-based operator [41, 42, 47] tend to exploit cross-view complementarity while overlooking their mutual interference, ultimately leading to degraded model performance [4]. For instance, when determining the adjacency between r_1 and r_2 , introducing rasterized view can interfere with the clear traversability cues provided by the serialized view, misleading actual topology inference. Therefore, *how to jointly model and effectively fuse serialized and rasterized views of trajectories* is a critical challenge.

Limitation 2: Keypoint-based inferring strategy tends to misjudge keypoint connectivity in dense road areas. Existing SOTA methods [30, 42] operate in a keypoint-centric way for topology inference, where keypoints serve as the sole map elements to extract for connectivity prediction. This keypoint extraction process, analogous to small target detection in computer vision [46, 49], is inherently constrained by a limited receptive field [21]. This limitation makes existing methods prone to generate connectivity errors in dense road areas. As illustrated in Figure 1 (c), a typical parallel road scenario contains densely distributed keypoints with highly similar spatial features (position and direction). Due to the limited context-aware capability at the road level, existing methods produce two types of errors: inferring a spurious shortcut (red arrow from kp_0 to kp_2 in Figure 1 (d)) between two separate roads and missing an actual link (dashed line from kp_1 to kp_2 in Figure 1 (d)) along the same road. To address this limitation, it is

essential to incorporate road-level context beyond local keypoints. Therefore, *how to formulate a context-aware strategy for inferring road topology* remains an unsolved challenge.

To solve these limitations, we propose D²Map, a Dual-view Map Inference Framework via Primal-Dual Graphs Co-generation. To precisely encode road adjacency, we first introduce the serialized trajectory view as a complement to the rasterized view. We then devise a Mixture of Expert (MoE) based strategy-adaptive fusion module to effectively incorporate the two-view representations. By employing a dynamic routing mechanism that adaptively selects and executes the optimal fusion operator (e.g., view disabling or gated fusion), our design mitigates inter-view interference and ensures the integration of complementary information, thereby generating comprehensive map element embeddings. To eliminate keypoint-level connectivity errors, we reformulate the process of map inference from a primal graph generation manner to a primal-dual graphs co-generation manner. In the dual graph, the nodes explicitly represent entire roads, shifting the modeling focus from local keypoints to holistic road structures. This road-centric modeling inherently facilitates broader contextual awareness. We implement this via a dual-branch architecture that jointly infers both graphs, in which a cross-graph consistency loss is designed to ensure the geometric alignment between the two graphs during model training, leading to more accurate topology inference.

Overall, our main contributions are summarized as follows:

- To improve road topological accuracy in complex map inference scenarios, we propose D²Map, a novel framework that introduces adaptive dual-view fusion and primal-dual graphs co-generation.
- We incorporate both serialized and rasterized view of trajectories for accurate map element embedding. Through an MoE-based strategy-adaptive fusion module, we ensure that the two views complement each other while avoiding mutual interference.
- We reformulate road map modeling from a primal graph to primal and dual graphs. Building on this extension, a dedicated primal-dual graphs co-generation module is designed to jointly infer both graphs while preserving their geometric consistency.
- A comprehensive evaluation conducted on two real-world datasets shows the superiority of D²Map in map inference and map updating under challenging scenarios.

2 Methods

2.1 Problem Statement

Definition 2.1 (Trajectory). A trajectory tr refers to a positional point sequence generated by a motor vehicle, denoted as $tr = \{p_1, \dots, p_i, \dots, p_n\}$, where $p_i = (l_i, t_i)$ represents a trajectory point at timestamp t_i , here l_i denotes p_i 's coordinate of longitude and latitude, i.e. $l_i = (lng_i, lat_i)$.

Definition 2.2 (Road Map). The road map is formulated as a primal graph and a dual graph to benefit robust topology inference.

The primal graph, denoted as $G_n = (V_n, E_n)$, is a directed graph. The vertex set V_n consists of all keypoints (namely, road intersections and dead ends). The edge set E_n denotes directed connections between the keypoints. In practice, E_n is represented by an adjacency matrix $M_n \in \mathbb{R}^{|V_n| \times |V_n|}$, where $M_n(i, j) = 1$ indicates that there exists a directed edge from v_{ni} to v_{nj} .

In the dual formulation, the dual graph is defined as $G_m = (V_m, E_m)$. Here, the vertex set V_m corresponds to the set of roads. The edge set E_m models the connectivity relationships between roads. Similarly to the primal modeling, E_m is represented by an adjacency matrix $M_m \in \mathbb{R}^{|V_m| \times |V_m|}$.

Problem Definition: Given a set of vehicle trajectories TR , the objective of this work is to infer the road map by jointly constructing its primal graph formulation G_n and dual graph formulation G_m .

2.2 Framework Overview

The overall architecture of D²Map is illustrated in Figure 2, which consists of three core steps: i) *Dual-view Feature Extraction*, ii) *Adaptive Dual-view Map Element Embedding* and iii) *Primal-Dual Graphs Co-Generation with Consistency Optimization*. *Dual-view Feature Extraction* first rasterizes and serializes the trajectory data into two-dimensional grids and trajectory point sequences. It then constructs behavioral features for each grid and trajectory point, ultimately generating the grid feature matrix and per-trajectory point feature sequences. *Adaptive Dual-view Map Element Embedding* initially generates grid representations and trajectory representations via a grid encoder and a trajectory encoder respectively. Subsequently, it employs a transformer decoder that integrates a hard-gated routing-based MoE layer to adaptively fuse the two view representations, yielding map element embeddings. *Primal-Dual Graphs Co-Generation with Consistency Optimization* employs a dual-branch architecture to jointly infer the primal graph and the dual graph from the map element embeddings. A cross-graph consistency loss is introduced to ensure geometric alignment between the two generated graphs during model training.

2.3 Dual-view Feature Extraction

Although rasterization-based methods [27, 42] have been widely adopted for map inference, they often struggle to accurately capture road adjacency relationships. In contrast, the serialized view of trajectories inherently encodes movement patterns that directly reflect the traversable relationships between roads. To leverage the strengths of both views, we model the trajectory data from two complementary perspectives: the rasterization view and the serialization view.

Grid Feature Extraction for the Rasterization View. Following [27, 42], we first partition the whole road map region into $I \times J$ fine-grained square grids Gr , with each grid gr having a side length of $gr.size$. Then we extract the behavioral features for each grid based on the trajectories passing through them: i) *Point Indicator*, denoting whether each grid contains a trajectory point, 1 if it does, 0 otherwise; ii) *Segment Indicator*, denoting whether each grid contains a trajectory segment. Two consecutive trajectory points are connected into a trajectory segment; iii) *Direction Distribution*, computed by mapping the angular direction between consecutive points into eight discrete bins; iv) *Average Speed*, derived from the mean distance between consecutive points normalized by time intervals. These features are organized into a grid feature matrix $\mathbf{H} \in \mathbb{R}^{11 \times I \times J}$ (the dimension of direction distribution equals to 8), which is subsequently partitioned into non-overlapping tiles following [27, 42] to meet the input size constraints of vision models. Thus each tile $\mathbf{T} \in \mathbb{R}^{11 \times H \times W}$ can be viewed as an image (H and W

respectively denote the height and the width of the tile) and each grid can be viewed as a pixel in the vision processing field.

Trajectory Feature Extraction for the Serialization View.

Given the trajectories of the tile, we extract per-point behavioral feature for each trajectory to formulate a spatial point feature sequence. However, the amounts of trajectories in a tile can be quite large, which is computationally expensive and impractical. To address this, we extract a subset of trajectories that reflect the moving behavior of all trajectories, termed representative trajectories. Inspired by the success of clustering algorithms in mining trajectory moving behavior [51], we first apply a grid-based trajectory clustering method proposed in [53] to extract clusters. The top- N_r trajectories closest to each cluster centroid are selected as the representative trajectories. Subsequently, we resample each trajectory to a fixed length of N_p equidistant points, ensuring a uniform sequence length for downstream modeling. Finally, we construct features for each point in these trajectories: i) position, relative coordinates within the tile; ii) direction, encoded as $(\cos\alpha, \sin\alpha)$, where α is the heading direction of the point. These features are organized as the point feature sequences of all representative trajectories within the tile $\mathbf{S} \in \mathbb{R}^{4 \times N_p \times N_r}$, where N_r denotes the number of representative trajectories in each tile, determined by N_r and the number of clusters (may vary across different tiles).

2.4 Adaptive Dual-view Map Element Embedding

This step aims to generate embeddings for keypoints and roads (refer to the map elements) within the tile from the extracted grid feature matrix and trajectory point feature sequences. Recent works adopt naive fusion methods like concatenated-based fusion [34] or attention-based fusion [44, 47], which focus on the complementarity between the multi-view features, ignoring the information interference, harming road adjacent relationship modeling. We design an MoE-based fusion strategy selection mechanism in map element embedding module to solve this limitation, aiming to dynamically adjust the fusion strategy across different scenarios to select and fuse dual-view representations. The process comprises three stages: (i) Multi-scale Grid Encoding, (ii) Neighboring-enhanced Trajectory Encoding, (iii) Fusion Strategy-adaptive Map Element Embedding.

Multi-scale Grid Encoding. Map elements vary in size [30], where major roads are wider and residential streets are narrower, posing a challenge for map element modeling. Motivated by the success of multi-scale representation learning in detecting objects of variable sizes [54], we extract multi-scale grid representations from the grid feature matrix $\mathbf{T}$. Following previous works [43], we first extract the initial grid representations at different scales $\{\mathbf{X}_{g_0}^l \in \mathbb{R}^{C \times \frac{H}{2^{l+1}} \times \frac{W}{2^{l+1}}}\}_{l=1}^4$ through the ResNet50 [14] and a 1×1 convolution layer. Then, we employ a deformable transformer encoder [54] to build multi-scale correlation of grid representations through layers of deformable attention, resulting in refined multi-scale grid representations $\{\mathbf{X}_g^l \in \mathbb{R}^{C \times \frac{H}{2^{l+1}} \times \frac{W}{2^{l+1}}}\}_{l=1}^4$.

Neighboring-enhanced Trajectory Encoding. Given $\mathbf{S} \in \mathbb{R}^{4 \times N_p \times N_r}$, the trajectory encoding module converts them into trajectory representations $\mathbf{X}_t \in \mathbb{R}^{C \times N_r}$. We treat each trajectory as a polyline, a consistent expression with the road. Considering the

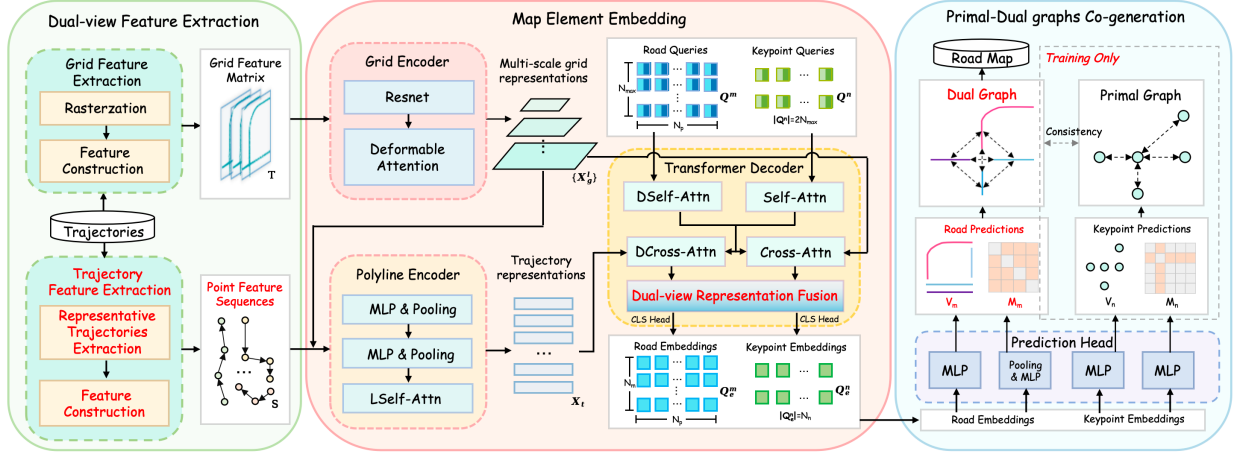


Figure 2: The framework of D²Map

inherent noise and low sampling rate of the trajectory data, a single trajectory cannot effectively reflect the geometric shape of the actual road it traveled. Therefore, we enhance each trajectory representation based on its neighbor grids. $\mathbf{S}$ is first fed into a multilayer perceptron network (MLP) to convert the per-point feature vector into the dense embeddings $\mathbf{X}_{t0} \in \mathbb{R}^{C \times N_p \times N_t}$. Then, for each point, we extract its corresponding multi-scale grid representations using a *grid sample* operator according to its position from $\{\mathbf{X}_g^l \in \mathbb{R}^{C \times \frac{H}{2^{l+1}} \times \frac{W}{2^{l+1}}}\}_{l=1}^4$. The extracted grid representations are concatenated and converted into the region-level contextual representation $\mathbf{X}_{tr} \in \mathbb{R}^{C \times N_p \times N_t}$ by the linear projection. $\mathbf{X}_{tr}$ is concatenated with $\mathbf{X}_{t0}$ to formulate the input of the polyline encoder $\mathbf{X}'_{t0} \in \mathbb{R}^{2C \times N_p \times N_t}$. We adopt a commonly adopted polyline encoder [31] to generate trajectory representations $\mathbf{X}_t \in \mathbb{R}^{C \times N_t}$.

Fusion Strategy-adaptive Map Element Embedding. Inspired by the success of the transformer decoder in representing polylines and keypoints [23, 43], we apply a transformer decoder to extract map element embeddings from the grid representations and trajectory representations. The decoder views each candidate map element as a learnable query, and then iteratively refines them by interacting them with grid representations and trajectory representations through several cascaded decoder layers, yielding candidate map element embeddings. A binary classification head then predicts the probability for each candidate to be valid. Positive instances are selected as the final map element embeddings.

Specifically, we follow the hierarchical query schema in [23] to formulate the road queries as a tensor $\mathbf{Q}^m \in \mathbb{R}^{C \times N_p \times N_{max}}$, where each of the N_{max} roads is modeled as a fixed-length, ordered sequence of N_p points. Here, N_{max} is a preset constant which is much greater than the number of map elements in the tiles. N_p equals the number of points in the resampled trajectory. The start and end point queries of each road in $\mathbf{Q}^m$ are copied to form the keypoint queries $\mathbf{Q}^n \in \mathbb{R}^{C \times 2N_{max}}$. Both $\mathbf{Q}^m$ and $\mathbf{Q}^n$ are then fed into the decoder. Each decoder layer incorporates two self-attention modules, two cross-attention modules, and a dual-view representation fusion module. We first feed road and keypoint queries into two self-attention modules, respectively interact them with each other

to facilitate the modeling of global road scenarios. For road query interaction, we follow [23] to utilize a decoupled self-attention mechanism to reduce computational cost. For keypoint queries, we adopt a vanilla self-attention layer to process. Then, the first cross-attention module employs the deformable attention [54] to interact the map element queries with the grid representations $\{\mathbf{X}_g^l\}_{l=1}^4$, yielding map element embeddings for the rasterization view ($\mathbf{Q}_g^m, \mathbf{Q}_g^n$). The second cross-attention module uses the vanilla attention to interact the map element queries with the trajectory representations $\mathbf{X}_t$, yielding map element embeddings for the serialization view ($\mathbf{Q}_t^m, \mathbf{Q}_t^n$). The above process is formulated as:

$$\begin{aligned} \mathbf{Q}'^m &= DSelfAttn(\mathbf{Q}^m); \mathbf{Q}'^n = SelfAttn(\mathbf{Q}^n) \\ (\mathbf{Q}_g^m, \mathbf{Q}_g^n) &= DCrossAttn(\mathbf{Q}'^m || \mathbf{Q}'^n, \{\mathbf{X}_g^l\}_{l=1}^4) \\ (\mathbf{Q}_t^m, \mathbf{Q}_t^n) &= CrossAttn(\mathbf{Q}'^m || \mathbf{Q}'^n, \mathbf{X}_t) \end{aligned} \quad (1)$$

where $DSelfAttn$ denotes the decoupled self-attention operator [23], $SelfAttn$ denotes the vanilla self-attention operator, $DCrossAttn$ denotes the deformable attention operator [54] and $CrossAttn$ denotes the vanilla cross-attention operator.

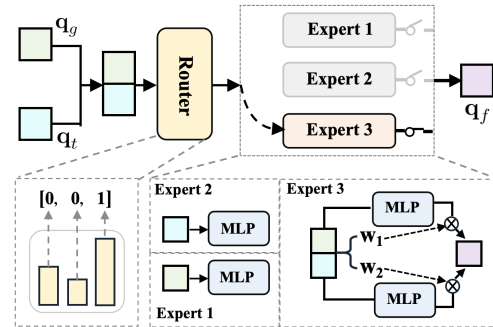


Figure 3: The Dual-view Representation Fusion Module

The dual-view representation fusion module aims to dynamically select the optimal fusion strategy to fuse the dual representations for map element embedding. The Mixture of Experts (MoE) network

has gained success in adaptively selecting specialized sub-networks [45]. Motivated by this, we adopt MoE to dynamically select the optimal fusion strategy to fuse the dual-view representations. As shown in Figure 3, for each map element query in rasterization view $\mathbf{q}_g$ and serialization view $\mathbf{q}_t$ (collected from $(\mathbf{Q}_g^m, \mathbf{Q}_g^n)$ and $(\mathbf{Q}_t^m, \mathbf{Q}_t^n)$), we define three fusion strategies and their corresponding experts: i) using only $\mathbf{q}_g$, with an MLP as the expert $\mathbf{q}_{e1} = \text{MLP}_g(\mathbf{q}_g)$. ii) using only $\mathbf{q}_t$, with an MLP as the expert $\mathbf{q}_{e2} = \text{MLP}_t(\mathbf{q}_t)$. and iii) adaptively fusing $\mathbf{q}_g$ and $\mathbf{q}_t$, with a gated fusion operator as the expert $\mathbf{q}_{e3} = w_1 \cdot \text{MLP}_g(\mathbf{q}_g) + w_2 \cdot \text{MLP}_t(\mathbf{q}_t)$, where $(w_1, w_2) = \sigma(\mathbf{W}_1[\mathbf{q}_g; \mathbf{q}_t])$. Initially, we concatenate and feed $(\mathbf{q}_g, \mathbf{q}_t)$ into a hard-gated router to obtain the routing vector $\mathbf{g} \in \mathbb{R}^3$. The routing vector subsequently guides the selection of experts and the selected expert generates the dual-view enhanced map element embedding $\mathbf{q}_f$. The process is defined as follows:

$$\mathbf{g} = \text{Onehot}(\text{argmax}_i(\text{MLP}(\mathbf{q}_g \parallel \mathbf{q}_t))); \mathbf{q}_f = \sum_{i=1}^3 g_i \cdot \mathbf{q}_{ei}, \quad (2)$$

where MLP denotes the MLP function, and $\text{Onehot}(\text{argmax}_i(\cdot))$ denotes the hard gated router, converting the index of the expert with the highest score into a one-hot tensor. Notice that the discrete decisions given by $\text{Onehot}(\text{argmax}_i(\cdot))$ are non-differentiable in training, we discuss and solve this problem in Appendix A.3.

The above process adaptively selects the optimal fusion strategy for fusing the two-view representations, yielding dual-view enhanced candidate map element embeddings $(\mathbf{Q}_f^m, \mathbf{Q}_f^n)$. Then, the road embeddings $\mathbf{Q}_f^m \in \mathbb{R}^{C \times N_p \times N_{max}}$ are flattened along the former two dimensions and fed into a classification head. Positive ones are selected as valid road embeddings $\mathbf{Q}_e^m \in \mathbb{R}^{C \times N_p \times N_m}$. Similarly, the keypoint embeddings $\mathbf{Q}_f^n \in \mathbb{R}^{C \times 2N_{max}}$ are fed into another classification head to obtain the valid keypoint embeddings $\mathbf{Q}_e^n \in \mathbb{R}^{C \times N_n}$. N_m and N_n respectively denotes the number of valid road embeddings and keypoint embeddings.

2.5 Primal-Dual Graphs Co-Generation with Consistency Optimization

Previous works primarily link the extracted keypoints to construct the map. They suffer from limited context awareness, leading to poor connectivity results, i.e., generating spurious shortcuts or fragmented roads. To this end, we revisit the modeling of the map, reformulating the road map from a primal graph to primal and dual graphs. In the dual graph, roads serve as nodes (as opposed to keypoints in the primal graph), and edges denote the connectivity between roads, providing broader context awareness. Based on this, we develop a *Primal-Dual Graphs Co-generation* module to infer the map, which first predicts the positions of map elements. Then, by predicting the relation of them, this module simultaneously generates the primal graph and the dual graph of the road map. Finally, in model optimization stage, a consistency-aware loss function is introduced to maintain the geometric consistency between the two generated graphs.

Topology Prediction. To obtain the positions of nodes for the primal graph and the dual graph, the road embeddings $\mathbf{Q}_e^m \in \mathbb{R}^{C \times N_p \times N_m}$ and the keypoint embeddings $\mathbf{Q}_e^n \in \mathbb{R}^{C \times N_n}$ are first fed into two MLPs to obtain their coordinates, termed $V_m \in \mathbb{R}^{2 \times N_p \times N_m}$

and $V_n \in \mathbb{R}^{2 \times N_n}$. Then, for edge prediction, we propose to directly predict the adjacency matrix from the map element embeddings. Specifically, we formulate it as a classification task. For any two road embeddings $\mathbf{q}_i^m \in \mathbb{R}^{C \times N_p}$ and $\mathbf{q}_j^m \in \mathbb{R}^{C \times N_p}$, we first apply mean-pooling along the point dimension to extract the road-wise embeddings. The two road-wise embeddings are concatenated and fed into an MLP-based classifier to obtain the probability of their connectivity, resulting in the road adjacency matrix $M_m \in \mathbb{R}^{N_m \times N_m}$. Similarly, we predict the keypoint adjacency matrix $M_n \in \mathbb{R}^{N_n \times N_n}$. For any two keypoint embeddings $\mathbf{q}_i^n \in \mathbb{R}^C$ and $\mathbf{q}_j^n \in \mathbb{R}^C$, we concatenate them and employ another MLP-based classifier to predict their linking relationship. This process is defined as:

$$M_{m(i,j)} = \text{Cls}_{rm}(\mu(\mathbf{q}_i^m) \parallel \mu(\mathbf{q}_j^m)); M_{n(i,j)} = \text{Cls}_{rn}(\mathbf{q}_i^n \parallel \mathbf{q}_j^n) \quad (3)$$

where μ denotes the mean-pooling operator and Cls_* denotes the classifier, consists of an MLP with a sigmoid activation function. Through the above procedure, we construct the primal graph (V_m, M_m) and the dual graph (V_n, M_n) for each tile. During inference, we adopt the dual graph as the output road map due to its superior contextual awareness.

Consistency-aware Model Optimization. In this stage, we describe how to train the map element embedding module and the map inference module. Directly treating the inference of the primal map and the dual map as two independent tasks can lead to structurally inconsistent outputs. For instance, a road intersection in the primal map should correspond to the convergence of the two associated roads in the dual map. However, in the absence of an explicit consistency constraint, it cannot align the inferred keypoint with the endpoints of its corresponding roads, preventing them from converging at an expected position. To this end, we introduce a consistency-aware loss function into the overall loss function to maintain the node position consistency of the two generated maps.

For each tile, we first extract the labels from the original ground truth map, including: i) ground truth roads $\hat{V}_m \in \mathbb{R}^{2 \times N_p \times \hat{N}_m}$, which are obtained by interpolating N_p points at equal intervals along the original road. ii) ground truth road adjacency matrix $\hat{M}_m \in \mathbb{R}^{\hat{N}_m \times \hat{N}_m}$. iii) ground truth keypoints $\hat{V}_n \in \mathbb{R}^{2 \times \hat{N}_n}$, including road intersections and dead ends. iv) ground truth keypoint adjacency matrix $\hat{M}_n \in \mathbb{R}^{\hat{N}_n \times \hat{N}_n}$. $\hat{N}_m$ and $\hat{N}_n$ denote the number of ground truth roads and keypoints. Based on the predictions and the labels, we introduce multi-task learning loss to train the D²Map in an end-to-end manner $\mathcal{L} = \lambda_1 \mathcal{L}_r + \lambda_2 \mathcal{L}_{rt} + \lambda_3 \mathcal{L}_p + \lambda_4 \mathcal{L}_{pt} + \lambda_5 \mathcal{L}_c$, where $\mathcal{L}_r$, $\mathcal{L}_{rt}$ denotes the loss for road position prediction and road topology prediction, $\mathcal{L}_p$, $\mathcal{L}_{pt}$ denotes the loss for keypoint position prediction and keypoint topology prediction, $\mathcal{L}_c$ denotes the position consistency loss. $\lambda_1, \lambda_2, \lambda_3, \lambda_4, \lambda_5$ are hyperparameters to adjust the weights of different losses.

For road position prediction, the computation of $\mathcal{L}_r$ is identical to the process in [23]. For road topology prediction, we treat it as a binary classification task, similarly to the scene graph generation task [32]. Due to the sparsity of the ground truth adjacency matrix, we randomly sample N_{neg} negative links from $\hat{M}_m$ and combine them with the positive links to form the input of the cross entropy loss, yielding $\mathcal{L}_{rt}$. For keypoint position prediction, we view it as an object detection task, where only the center of each object is retained and the width, height of each object are ignored. To

this end, we apply the same procedure in [43] to compute $\mathcal{L}_p$. For keypoint topology prediction, the computation of $\mathcal{L}_{pt}$ is the same as $\mathcal{L}_{rt}$. Due to the space limitation, we recommend readers visit these references [23, 32, 43] for a more detailed description.

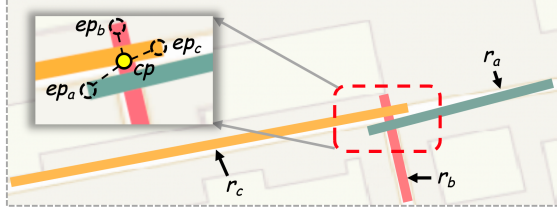


Figure 4: The example of endpoints inconsistency

To compute the consistency loss $\mathcal{L}_c$, the first step is to establish correspondence between the two generated graphs. Specifically, we first identify the convergence points of the linked roads from the predicted dual graph, where each convergence point aligns with several endpoints of the roads. As shown in Figure 4, roads r_a , r_b and r_c are inferred as connected. Due to that their predicted endpoints (ep_a , ep_b , ep_c) may not coincide spatially, the average position of these endpoints is viewed as the estimated position of the convergence point, yielding the convergence point set $\{cp_i\}$. Simultaneously, we extract the predicted road intersections $\{rp_i\}$ from V_n based on the constraint that the node degree should be greater than 2. Then, given all convergence points and road intersections, we employ the Hungarian algorithm to find the optimal matching between them. The matching cost is defined as the Manhattan distance between each convergence point and each road intersection. Let $\mathcal{M} = \{\pi_i\}_{i=0}^{|\mathcal{M}|}$, $\pi_i = (cp_i, rp_i, \Gamma_i)$ be the matching result, where cp_i , rp_i , Γ_i are respectively the convergence point, road intersection and the aligned endpoints set of cp_i . $\mathcal{L}_c$ is computed as:

$$\mathcal{L}_c = \frac{1}{|\mathcal{M}|} \sum_{\pi_i \in \mathcal{M}} (\|cp_i - rp_i\|_1 + \frac{1}{|\Gamma_i|} \sum_{ep \in \Gamma_i} \|cp_i - ep\|_1) \quad (4)$$

where $\|\cdot\|_1$ denotes the Manhattan distance and ep denotes the aligned endpoint. The former term encourages the position consistency between the intersections in the primal graph and the road convergence points in the dual graph, while the latter ensures alignment of the endpoints of linked roads in the dual graph.

3 Experiments

3.1 Experiment Settings

Datasets. We use two trajectory datasets in the experiments, i.e., **WX20** and **SH18**. **WX20** [42] is a publicly available taxi trajectory dataset in Wuxi city over a period of 14 days. **SH18**¹ is a public taxi trajectory dataset in Shanghai city over a period of 3 days. The ground truth road map is fetched from *OpenStreetMap*², which is an editable road map created by the public. The detailed description of datasets is given in Appendix A.1.

Baselines. We compare D²Map with 9 representative methods in the field of map inference from trajectories, including 5

heuristic-based methods (*Cao09*, *Biagioni12*, *Kharita18*, *RoadRunner18*, *Huang19*), and 4 learning-based methods (*DeepMG20*, *Ringnet22*, *CDMap25*, *DGMap25*). These baselines represent representative approaches across different categories of map inference methods mentioned in 4. Besides, our task can be solved by employing existing methods in the fields of map inference from satellite images [30]. To this end, we also compare D²Map with the following methods, *Sat2Graph20*, *TDRoad22* and *Road2graph24*. More details about the baselines are provided in Appendix A.2. The implementation details of D²Map are provided in Appendix A.3.

Evaluation metrics. In the evaluation, we adopt *TOPO* [2] and *Average Path Length Similarity (APLS)* [35] to verify the accuracy of the inferred road map, as these metrics are robust to both topological errors and positional errors [16]. *TOPO* aims to evaluate the similarity between the subgraphs extracted from the inferred road map and the ground truth road map. The sampling points of two subgraphs are matched according to the matching distance threshold d_t . We set d_t to 5 meters, 10 meters and 15 meters to execute a comprehensive evaluation. The precision, recall, and F1-score of the matching results are reported as metrics. *APLS* measures the difference in shortest paths that are randomly sampled from the inferred road map and the ground truth road map. The similarity in the lengths of the two shortest paths is chosen as the metric.

3.2 Performance Evaluation

Overall Performance. To quantitatively evaluate our proposed D²Map, we compare the accuracy of D²Map with the 9 baselines in the fields of map inference from trajectories. The comparison with methods in the fields of map inference from satellite images are presented in Appendix B. As reported in Table 1, we make the following observations: i) D²Map performs the best in terms of *TOPO* and *APLS* on map inference task in two real-world datasets, outperforming the best baseline by 8.09%, 11.44% in F1-score ($d_t = 10m$) and 7.96%, 15.37% in *APLS* on WX20, SH18. Compared to the existing SOTA baselines (primarily learning-based methods) that adopt a discrete rasterized representation of trajectories, D²Map introduces a serialized representation of trajectories and integrates it with the rasterized view. By modeling and fusing dual-view feature embeddings, D²Map effectively models the structural relationships among roads. Additionally, by modeling the road map as primal and dual graphs, D²Map avoids the limitations of existing keypoint-based methods. Through jointly inferring both graphs, D²Map enables more accurate road topology inference. ii) Heuristic-based methods generally perform worse than learning-based methods. Despite *Biagioni* and *RoadRunner* achieve a high precision, they miss to identify too many roads, leading to poor Recall and F1-score. In 4 learning-based methods, both *DGMap* and *CDMap* are keypoint-based methods and are the best-performing baselines. This is because, keypoint-based methods explicitly model the road topology via keypoint connectivity, whereas segmentation-based approaches, such as *DeepMG*, model the road map as discrete grids, lacking a direct characterization of topological structure.

Visualization Performance. To further qualitatively evaluate D²Map, we visualize typical cases under challenge settings in Figure 5, including the overpass (*scenario a*), and the parallel roads

¹<https://soda.data.sh.gov.cn/competitionData.html>

²<https://www.openstreetmap.org/>

Table 1: Performance Evaluation for Different Methods on Map Inference Task

Dataset	Type	Methods	Evaluation Criteria									APLS
			TOPO									
			$d_t=5m$			$d_t=10m$			$d_t=15m$			
Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score				
WX20	Heuristic	Cao09	0.3781	0.2745	0.3180	0.6532	0.5061	0.5703	0.7622	0.5028	0.6732	0.4464
		Biagioni12	0.5061	0.0757	0.1223	0.8792	0.1796	0.2726	0.9538	0.2445	0.3559	0.2108
		Kharita18	0.4497	0.2549	0.2912	0.5631	0.4409	0.4945	0.7232	0.5845	0.6465	0.3351
		RoadRunner18	0.4669	0.2748	0.3460	0.7783	0.4902	0.6016	0.9073	0.5954	0.7197	0.5218
		Huang19	0.4304	0.2339	0.3031	0.7543	0.4422	0.5575	0.8961	0.5482	0.6803	0.4266
	Learning	Deepmg20	0.4370	0.2974	0.3540	0.7548	0.5497	0.6361	0.8798	0.6722	0.7621	0.5515
		Ringnet22	0.5036	0.2516	0.3356	0.8497	0.4807	0.6140	0.9316	0.5542	0.6950	0.5227
		CDMap25	0.4801	0.3312	0.3920	0.8333	0.6275	0.7159	0.9081	0.7028	0.7923	0.5935
		DGMap25	0.4702	0.3608	0.4083	0.7672	0.6270	0.6901	0.8835	0.7451	0.8084	0.6272
		D ² Map	0.5412	0.4294	0.4789	0.8509	0.7094	0.7738	0.9382	0.7988	0.8629	0.6771
SH18	Heuristic	Cao09	0.3445	0.2530	0.2917	0.6063	0.4691	0.5290	0.7142	0.5655	0.6312	0.3145
		Biagioni12	0.3869	0.1449	0.2108	0.6636	0.2594	0.3730	0.7861	0.3178	0.4526	0.2348
		Kharita18	0.2833	0.2678	0.2753	0.4834	0.4572	0.4700	0.5873	0.5652	0.5747	0.2941
		RoadRunner18	0.5338	0.2364	0.3277	0.8493	0.4107	0.5537	0.9138	0.4599	0.6118	0.3503
		Huang19	0.4384	0.2290	0.3008	0.7625	0.4314	0.5510	0.8977	0.5371	0.6721	0.4279
	Learning	Deepmg20	0.4437	0.2821	0.3461	0.7573	0.5152	0.6131	0.8870	0.6338	0.7393	0.5169
		Ringnet22	0.5036	0.2519	0.3356	0.8320	0.4660	0.5974	0.9348	0.5551	0.6966	0.4648
		CDMap25	0.4688	0.3321	0.3887	0.7732	0.6006	0.6760	0.8899	0.7242	0.7985	0.5703
		DGMap25	0.4671	0.3711	0.4136	0.7745	0.6286	0.6940	0.8872	0.7505	0.8131	0.5764
		D ² Map	0.5333	0.3997	0.4570	0.8672	0.6980	0.7734	0.9484	0.7935	0.8640	0.6650

(scenario b). More visualization results are given in Appendix B. *Scenario a* shows an example of the overpass. Since rasterization-based methods confuse adjacent relationships between roads, they mistakenly identify overpass roads as a crossroad (marked by red circles). In contrast, D²Map explicitly models the continuity between roads through the serialized representation, allowing it to successfully recognize the correct topology. *Scenario b* presents a typical parallel road topology, comprising four roads that are close to each other. *CDMap* and *DGMap* produce both fragmented roads and spurious shortcuts (marked by red rectangles). Since the Primal-Dual Graphs Co-generation module jointly models roads and keypoints in map inference, D²Map generates topologically continuous roads.

Evaluation of Hard Cases. To quantitatively assess the performance of existing methods and D²Map in challenging scenarios, we construct two dedicated test subsets for each dataset, *Parallel Roads* subset and *Overpass* subset. The construction process is illustrated in Appendix A.4. We report the F1-score across all tiles in these subsets, as shown in Figure 6. Thanks to the design of Adaptive Dual-view Map Element Embedding module and the Primal-Dual Graphs Co-generation module, our method achieves an accuracy improvement of approximately 10% in F1-score on these subsets, which demonstrates D²Map’s capability for map inference in complex scenarios.

Impact on Map Updating. One of the most critical applications of map inference is **map updating**—a fundamental requirement for modern web-based location services such as navigation, ride-hailing and logistics. To this end, the map updating framework in MapFuse [36] is selected to execute the fair comparison. We integrate both D²Map and the baselines into this framework, and evaluate the accuracy of the generated updated maps. We randomly remove a certain proportion of roads ($\theta \in \{30\%, 50\%, 70\%\}$) from the ground truth road map to simulate an outdated map, and then calculate the TOPO metric of the updated road map against the ground truth.

Figure 7 illustrates the F1-score of TOPO metric on WX20 dataset. We observed that our proposed D²Map achieves the highest F1-scores under all θ settings, demonstrating its superiority in accurately updating road maps. These results underscore the value of our approach for web-based services, where precise map updates are critical. The result on SH18 dataset is presented in Appendix B.

3.3 Model Analyses

Ablation Studies. To ascertain the impact of different modules within D²Map, we perform ablation studies focusing on the following components: i) *w/o TEM*: Remove the trajectory encoding and its related components; ii) *Gated Fusion*: Replace the MoE-based fusion with vanilla gated fusion; iii) *w/o KP*: Remove the keypoint topology prediction head and its related components; iv) *w/o CL*: Remove the consistency loss during the model training.

The results are summarized in Table 2. We observe that *w/o TEM* and *w/o KP* lead to significant performance degradation, confirming the critical role of trajectory serialization and the effectiveness of primal-dual graph modeling in capturing road network topology. *Gated Fusion* performs worse than D²Map, demonstrating the necessity of a more comprehensive fusion for representing map elements. Furthermore, *w/o CL* performs better than *w/o KP* but worse than D²Map, indicating that enforcing consistency between the primal and dual graphs during training improves the accuracy of topological predictions. The parameter sensitivity analysis and the expert weight visualizations are presented in the Appendix B.

4 Related Work

4.1 Heuristic-based Map Inference Methods

Early studies on heuristic-based map inference fall into four main categories: i) *Clustering-based* methods identify vertices or edges through spatial clustering and connect them based on trajectory density [7, 10, 22, 33]. ii) *Density-based* methods transform trajectories into density images, apply kernel density estimation to extract

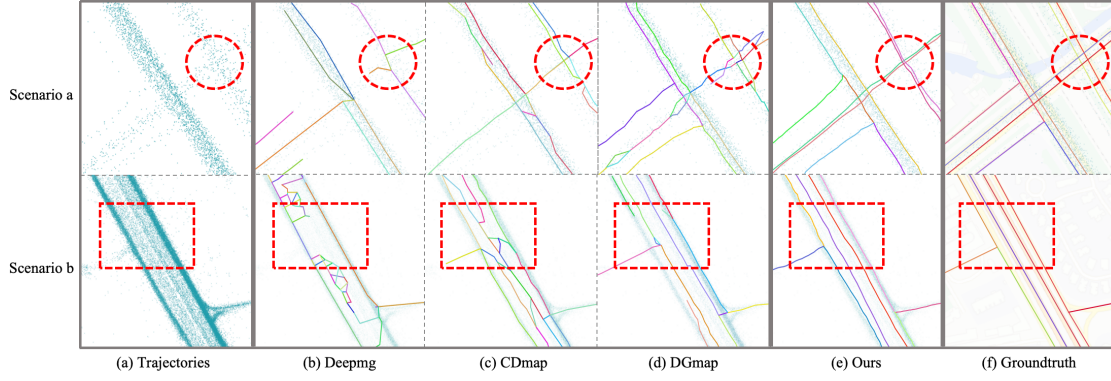


Figure 5: Visualization of inferred road maps

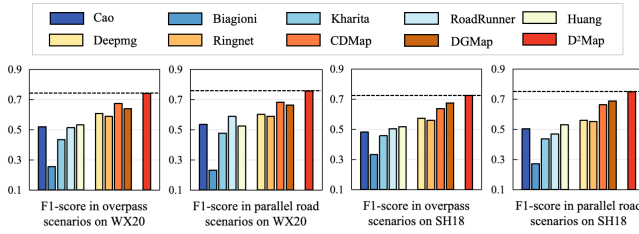


Figure 6: F1-score for Hard Cases

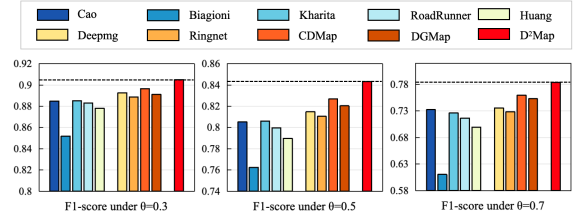


Figure 7: F1-score on WX20 for Map Updating

Table 2: Ablation Study on Two Datasets

Dataset	Methods	Evaluation Criteria			
		TOPO($d_t=10m$)			APLS
		Precision	Recall	F1-score	
WX20	w/o TEM	0.8466	0.6594	0.7413	0.6401
	Gated Fusion	0.8512	0.6882	0.7611	0.6643
	w/o KP	0.8453	0.6784	0.7527	0.6554
	w/o CL	0.8532	0.6852	0.7600	0.6689
	D²Map	0.8509	0.7094	0.7738	0.6771
SH18	w/o TEM	0.8613	0.6770	0.7581	0.6438
	Gated Fusion	0.8566	0.6949	0.7623	0.6539
	w/o KP	0.8507	0.6863	0.7597	0.6581
	w/o CL	0.8581	0.6949	0.7679	0.6605
	D²Map	0.8672	0.6980	0.7734	0.6650

road skeletons, and then convert these into graphs [3, 8, 9]. iii) *Intersection-linking-based* methods detect road intersections using cues like directional diversity and speed variation, then link them to form the road network [12, 18, 20, 29]. iv) *Incremental insertion-based* methods begin with an empty map and iteratively expand it by inserting and merging new trajectories [1, 5, 15]. Since the performance of all these approaches heavily relies on manually predefined parameters, they generally suffer from limited accuracy and poor scalability.

4.2 Learning-based Map Inference Methods

Learning-based methods [11, 27, 30, 42] first rasterize trajectories into density images and then infer road maps from these images. These methods are primarily divided into two categories: i) *Segmentation-based* methods [11, 27] use a segmentation head to

identify pixels belonging to road centerlines, followed by heuristic post-processing to convert the pixel masks into a graph structure. ii) *Keypoint-based* methods [30, 42] first detect pixels corresponding to keypoints sampled along roads and then connect them via a topology prediction head. However, by reducing trajectories to pixel grids, rasterization-based methods fail to capture structural relationships between roads, leading to topological errors in non-planar scenario. Moreover, keypoint-centric modeling suffers from limited contextual awareness, often resulting in spurious shortcuts and fragmented roads in dense road areas.

5 Conclusion

To improve topological accuracy in complex road scenarios, we propose *D²Map*, a dual-view map inference framework via primal-dual graphs co-generation. By fusing serialized and rasterized trajectory representations and jointly inferring the keypoint-centric primal graph and road-centric dual graph, *D²Map* significantly enhances structural reasoning and connectivity prediction. Experiments on two real-world datasets demonstrate the superior performance of *D²Map* in handling complex road topologies. In the future, we plan to improve generalization to few-shot road topologies and incorporate multimodal data, such as satellite imagery and aerial videos, to further boost inference robustness in challenging environments.

Acknowledgments

This work was supported by NSFC under Grant No. 62461146205.

References

- [1] Mahmuda Ahmed and Carola Wenk. 2012. Constructing street networks from GPS trajectories. In *ESA*. 60–71.
- [2] James Biagioni and Jakob Eriksson. 2012. Inferring road maps from global positioning system traces: Survey and comparative evaluation. *Transportation research record* 2291, 1 (2012), 61–71.
- [3] James Biagioni and Jakob Eriksson. 2012. Map inference in the face of noise and disparity. In *SIGSPATIAL*. 79–88.
- [4] Bing Cao, Yiming Sun, Pengfei Zhu, and Qinghua Hu. 2023. Multi-modal gated mixture of local-to-global experts for dynamic image fusion. In *CVPR*. 2355–2364.
- [5] Lili Cao and John Krumm. 2009. From GPS traces to a routable road map. In *SIGSPATIAL*. 3–12.
- [6] Pingfu Chao, Wen Hua, Rui Mao, Jiajie Xu, and Xiaofang Zhou. 2020. A survey and quantitative study on map inference algorithms from gps trajectories. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2020), 15–28.
- [7] Chen Chen, Cewu Lu, Qixing Huang, Qiang Yang, Dimitrios Gunopulos, and Leonidas Guibas. 2016. City-scale map creation and updating using GPS collections. In *SIGKDD*. 1465–1474.
- [8] Jonathan J Davies, Alastair R Beresford, and Andy Hopper. 2006. Scalable, distributed, real-time map generation. *IEEE Pervasive Computing* 5, 4 (2006), 47–54.
- [9] Tamal K Dey, Jiayuan Wang, and Yusu Wang. 2017. Improved road network reconstruction using discrete morse theory. In *SIGSPATIAL*. 1–4.
- [10] Stefan Edelkamp and Stefan Schrödl. 2003. Route planning and map inference with global positioning traces. In *Computer science in perspective*. 128–151.
- [11] Emre Eftelioglu, Ravi Garg, Vaibhav Kango, Chintan Gohil, and Amber Roy Chowdhury. 2022. RING-Net: Road inference from gps trajectories using a deep segmentation network. In *SIGSPATIAL Workshop*. 17–26.
- [12] Alireza Fathi and John Krumm. 2010. Detecting road intersections from GPS traces. In *GIScience*. 56–69.
- [13] Chenjuan Guo, Bin Yang, Jilin Hu, and Christian Jensen. 2018. Learning to route with sparse trajectory sets. In *ICDE*. IEEE, 1073–1084.
- [14] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *CVPR*. 770–778.
- [15] Songtao He, Favven Bastani, Sofiane Abbar, Mohammad Alizadeh, Hari Balakrishnan, Sanjay Chawla, and Sam Madden. 2018. RoadRunner: improving the precision of road network inference from GPS trajectories. In *SIGSPATIAL*. 3–12.
- [16] Songtao He, Favven Bastani, Satvat Jagwani, Mohammad Alizadeh, Hari Balakrishnan, Sanjay Chawla, Mohamed M Elsharif, Samuel Madden, and Mohammad Amin Sadeghi. 2020. Sat2graph: Road graph extraction through graph-tensor encoding. In *ECCV*. Springer, 51–67.
- [17] Yang He, Ravi Garg, and Amber Roy Chowdhury. 2022. Td-road: top-down road network extraction with holistic graph construction. In *ECCV*. Springer, 562–577.
- [18] Yourong Huang, Zhu Xiao, Xiaoyu Yu, Dong Wang, Vincent Havyarimana, and Jing Bai. 2019. Road network construction with complex intersections based on sparsely sampled private car trajectory data. *TKDD* 13, 3 (2019), 1–28.
- [19] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* (2016).
- [20] Sophia Karagiorgou and Dieter Pfoser. 2012. On vehicle tracking data-based road network generation. In *SIGSPATIAL*. 89–98.
- [21] Boyang Li, Chao Xiao, Longguang Wang, Yingqian Wang, Zaiping Lin, Miao Li, Wei An, and Yulan Guo. 2022. Dense nested attention network for infrared small target detection. *IEEE Transactions on Image Processing* 32 (2022), 1745–1758.
- [22] Hengfeng Li, Lars Kulik, and Kotagiri Ramamohanarao. 2016. Automatic generation and validation of road maps from GPS trajectory data sets. In *CIKM*. 1523–1532.
- [23] Bencheng Liao, Shaoyu Chen, Yunchi Zhang, Bo Jiang, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. 2025. Maptrv2: An end-to-end framework for online vectorized hd map construction. *International Journal of Computer Vision* 133, 3 (2025), 1352–1374.
- [24] Yicheng Liu, Tianyuan Yuan, Yue Wang, Yilun Wang, and Hang Zhao. 2023. Vectormapnet: End-to-end vectorized hd map learning. In *ICML*. PMLR, 22352–22369.
- [25] Mashaal Musleh, Sofiane Abbar, Rade Stanojevic, and Mohamed Mokbel. 2021. QARTA: an ML-based system for accurate map services. *PVLDB* 14, 11 (2021), 2273–2282.
- [26] Sijie Ruan, Ruiyuan Li, Jie Bao, Tianfu He, and Yu Zheng. 2018. Cloudtp: A cloud-based flexible trajectory preprocessing framework. In *ICDE*. IEEE, 1601–1604.
- [27] Sijie Ruan, Cheng Long, Jie Bao, Chunyang Li, Zisheng Yu, Ruiyuan Li, Yuxuan Liang, Tianfu He, and Yu Zheng. 2020. Learning to generate maps from trajectories. In *AAAI*, Vol. 34. 890–897.
- [28] Sijie Ruan, Cheng Long, Zhipeng Ma, Jie Bao, Tianfu He, Ruiyuan Li, Yiheng Chen, Shengnan Wu, and Yu Zheng. 2022. Service time prediction for delivery tasks via spatial meta-learning. In *SIGKDD*. 3829–3837.
- [29] Wenyi Shen, Wenyu Wu, Jiali Mao, Jie Chen, Shaosheng Cao, Lisheng Zhao, Aoying Zhou, and Lin Zhou. 2023. SAMI: A Shape-Aware Cycling Map Inference Framework for Designated Driving Service. In *ICDE*. IEEE, 3269–3281.
- [30] Yudong Shen, Wenyu Wu, Jiali Mao, Yixiao Tong, Guoping Liu, and Chaoya Wang. 2025. Bridging the Gap Between Sparsity and Redundancy: A Dual-Decoding Framework with Global Context for Map Inference. *arXiv:2509.11731 [cs.CV]* <https://arxiv.org/abs/2509.11731>
- [31] Shaoshuai Shi, Li Jiang, Dengxin Dai, and Bernt Schiele. 2022. Motion transformer with global intention localization and local movement refinement. *Neurips* 35 (2022), 6531–6543.
- [32] Suprosanna Shit, Rajat Koner, Bastian Wittmann, Johannes Paetzold, Ivan Ezhov, Hongwei Li, Jiazhen Pan, Sahand Sharifzadeh, Georgios Kaissis, Volker Trespe, et al. 2022. Relationformer: A unified framework for image-to-graph generation. In *ECCV*. Springer, 422–439.
- [33] Rade Stanojevic, Sofiane Abbar, Saravanan Thirumuruganathan, Sanjay Chawla, Fethi Filali, and Ahid Aleimat. 2018. Robust road map inference through network alignment of trajectories. In *SDM*. 135–143.
- [34] Tao Sun, Zonglin Di, Pengyu Che, Chun Liu, and Yin Wang. 2019. Leveraging crowdsourced GPS data for road extraction from aerial imagery. In *CVPR*. 7509–7518.
- [35] Adam Van Etten, Dave Lindenbaum, and Todd M Bacastow. 2018. Spacenet: A remote sensing dataset and challenge series. *arXiv preprint arXiv:1807.01232* (2018).
- [36] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR*.
- [37] Suyi Wang, Yusu Wang, and Yanjie Li. 2015. Efficient map reconstruction and augmentation via topological methods. In *SIGSPATIAL*. 1–10.
- [38] Shuliang Wang, Ziyu Wang, Sijie Ruan, Haoyu Han, Keqin Xiong, Hanning Yuan, Ziqiang Yuan, Guoqing Li, Jie Bao, and Yu Zheng. 2024. DelvMap: Completing Residential Roads in Maps Based on Couriers' Trajectories and Satellite Imagery. *IEEE Transactions on Geoscience and Remote Sensing* (2024).
- [39] Tao Wang, Jiali Mao, and Cheqing Jin. 2017. Hymu: A hybrid map updating framework. In *DASFAA*. 19–33.
- [40] Xiangheng Wang, Ziquan Fang, Chenglong Huang, Danlei Hu, Lu Chen, and Yunjun Gao. [n. d.]. GTR: A General, Multi-View, and Dynamic Framework for Trajectory Representation Learning. In *ICML*.
- [41] Hao Wu, Hanyuan Zhang, Xinyu Zhang, Weiwei Sun, Baihua Zheng, and Yun-jing Jiang. 2020. DeepDualMapper: A gated fusion network for automatic map extraction using aerial images and trajectories. In *AAAI*, Vol. 34. 1037–1045.
- [42] Wenyu Wu, Jiali Mao, Jiafan Liu, Yixiao Tong, Lisheng Zhao, Shaosheng Cao, Jilin Hu, Aoying Zhou, and Lin Zhou. 2025. CDMaP: Complementarity and Disparity-Aware Map Inference Quality Enhancement. In *ICDE*. IEEE Computer Society, 4156–4168.
- [43] Wenyu Wu, Wenyi Shen, Jiali Mao, Lisheng Zhao, Shaosheng Cao, Aoying Zhou, and Lin Zhou. 2023. PICT: Precision-enhanced Road Intersection Recognition Using Cycling Trajectories. In *ECMLPKDD*. Springer, 157–173.
- [44] Ronghui Xu, Hanyin Cheng, Chenjuan Guo, Hongfan Gao, Jilin Hu, Sean Bin Yang, and Bin Yang. 2025. Mm-path: Multi-modal, multi-granularity path representation learning. In *SIGKDD*. 1703–1714.
- [45] Zihui Xue and Radu Marculescu. 2023. Dynamic multimodal fusion. In *CVPR*. 2575–2584.
- [46] Bo Yang, Fengqian Li, Songliang Zhao, Wei Wang, Jun Luo, Huayan Pu, Mingliang Zhou, and Yangjun Pi. 2025. MTMLNet: Multi-Task Mutual Learning Network for Infrared Small Target Detection and Segmentation. *IEEE Transactions on Image Processing* (2025).
- [47] Haitao Yuan, Sai Wang, Zhifeng Bao, and Shangguang Wang. 2023. Automatic Road Extraction with Multi-Source Data Revisited: Completeness, Smoothness and Discrimination. *PVLDB* 16, 11 (2023), 3004–3017.
- [48] Yifan Zao, Zhengxia Zou, and Zhenwei Shi. 2024. Road Graph Extraction via Transformer and Topological Representation. *IEEE Geoscience and Remote Sensing Letters* (2024).
- [49] Mingjin Zhang, Rui Zhang, Yuxiang Yang, Haichen Bai, Jing Zhang, and Jie Guo. 2022. ISNet: Shape matters for infrared small target detection. In *CVPR*. 877–886.
- [50] Lisheng Zhao, Jiali Mao, Min Pu, Guoping Liu, Cheqing Jin, Weining Qian, Aoying Zhou, Xiang Wen, Runbo Hu, and Hua Chai. 2020. Automatic calibration of road intersection topology using trajectories. In *ICDE*. 1633–1644.
- [51] Yu Zheng. 2015. Trajectory data mining: an overview. *ACM Transactions on Intelligent Systems and Technology (TIST)* 6, 3 (2015), 1–41.
- [52] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Neurips* 35 (2022), 7103–7114.
- [53] Junhao Zhu, Tao Wang, Danlei Hu, Ziquan Fang, Lu Chen, Yunjun Gao, Tianyi Li, and Christian S Jensen. 2024. T-Assess: An Efficient Data Quality Assessment System Tailored for Trajectory Data. *PVLDB* 18, 3 (2024), 666–674.
- [54] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. 2020. Deformable detr: Deformable transformers for end-to-end object detection. *arXiv preprint arXiv:2010.04159* (2020).

A Experiment Settings

A.1 Dataset Description

Both **WX20** and **SH18** contain the attributes of Trajectory ID, Timestamp, Latitude, and Longitude. The detailed statistics of these datasets are given in Table 3, where the statistics of the test region are given in parentheses.

Table 3: Description of experimental datasets

Dataset	WX20	SH18
Time Span	200718-200801	180410-180412
Sampling Rate (s)	3	16
Size (km^2)	32×28 (6 $\times$ 6)	42×38 (6 $\times$ 6)
Point Count	3.6M (0.4M)	87.8M (7.0M)

A.2 Baselines

We provide the details of baseline methods in our experiments.

- **Cao** [5] designs a "clarification" step to adjust the position of trajectories by simulating attraction force between particles, followed by iteratively inserting trajectories into the current map to generate the road map.
- **Biagioni** [3] extracts the road skeleton based on Kernel Density Estimation, then utilizes the map-matching algorithm to remove low supported edges.
- **Kharita** [33] first infers the vertices from trajectory points using K-Means and then links the vertices based on the trajectories passing through them. Finally, the redundant edges are removed through the Graph Spanner algorithm.
- **RoadRunner** [15] consists of two stages: trace and merge. The former aims to predict new roads while the latter aims to merge these new roads into the current road map.
- **Huang** [18] clusters turning points using *DBSCAN* to get road intersections, then adopts a sliding window to scan over the trajectories passing through the intersections to generate the roads.
- **Deepmg** [27] converts the trajectory points into density-based images, and utilizes a UNet-like model to predict the road centerline mask. Then, it extracts and refines the road map through a heuristic graph refinement strategy.
- **Ringnet** [11] incorporates self-attention mechanism into D-LinkNet to predict the centerline mask. Then, it designs a vector conversion strategy to generate the road map.
- **CDMap** [42] designs a dual-branch architecture to respectively encode point-wise features and segment-wise features to generate grid embeddings, then predicts the road keypoints and their relationships from them based on graph tensor decoding.
- **DGMap** [30] designs a mask-enhanced keypoint extraction module to identify road keypoints from the density-based images, then a contextual-aware relation prediction module is employed to predict the connectivity between keypoints.
- **Sat2graph** [16] utilizes a DLA network to infer a 17-dimensional graph tensor, which is then decoded into the road map via a handcrafted decoding module.
- **TDRoad** [17] detects road keypoint first, then employs an MLP-based connectivity prediction model to predict their connectivity.
- **Road2graph** [48] utilizes a transformer-based model to incorporate multi-scale contextual information while predicting the graph tensor, then design a refined graph decoding strategy to convert the tensor to the road map.

A.3 Implementations

We implement D²Map with PyTorch 1.13.1. We run experiments on a machine with a 112-core CPU and an NVIDIA Tesla A800 GPU. Before extracting features for each grid, we apply preprocessing algorithms to raw trajectories, including trajectory segmentation and denoising in [50]. Since OpenStreetMap contains roads that are rarely or never traversed by vehicles, we remove any edge with fewer than 5 associated trajectories in both datasets, using the map-matching algorithm from [26]. We take the trajectories outside the test region as the training and validation data. To enlarge the number of training samples for stable training, we follow [43] to employ the sliding window model to obtain the training samples. Considering the diversity of road orientation distribution, we adopt random horizontal flipping, vertical flipping and horizontal then vertical flipping for data augmentation in training. The above strategies are also employed for our learning-based competitors to ensure fairness.

The height and width of each tile are both set to 256. The grid size *gr.size* is set to 1 meter, which follows the standard choice in early works [16]. We set the embedding dimensionality *C* to 256. In representative trajectories extraction, the grid resolution in trajectory clustering is set to 8, and the number of nearest trajectories N_r in clustering is set to 5. The length of each resampled trajectory and each road query are determined by the parameter sensitivity analysis. The preset number of candidate road queries N_{max} is set to 200. During model optimization, the loss weights $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are all set to 1. The selection of λ_5 is determined by the parameter sensitivity analysis. The number of negative links N_{neg} is set to 40, identical to [32]. We train D²Map using the AdamW optimizer and set the weight decay as 0.01. The learning rate is initialized to 2e-4 and decayed by half after every 10 epochs. Different from soft-gated MoEs [52] where the output is a weighted sum of expert networks, our designed hard-gated router in representation fusion module is non-differentiable because of the *onehot*($\cdot$) operator. To avoid this problem, we train the Mixture-of-Experts (MoE) layer using the Gumbel-softmax and the straight-through techniques [19], and a warmup schedule that gradually transitions from dense to sparse routing in the initial 5 training epochs.

A.4 Test Subset Construction

We construct two dedicated test subsets for each dataset: i) *Parallel Roads* subset, consisting of tiles that contain parallel roads. A tile is considered to contain parallel roads if there exists a pair of roads such that the distance between their closest points is less than 10 meters and they connect to different vertices. ii) *Overpass* subset, including tiles that contain stacking roads. A tile is considered to contain stacking roads if any two roads overlap in projection but are not topologically connected.

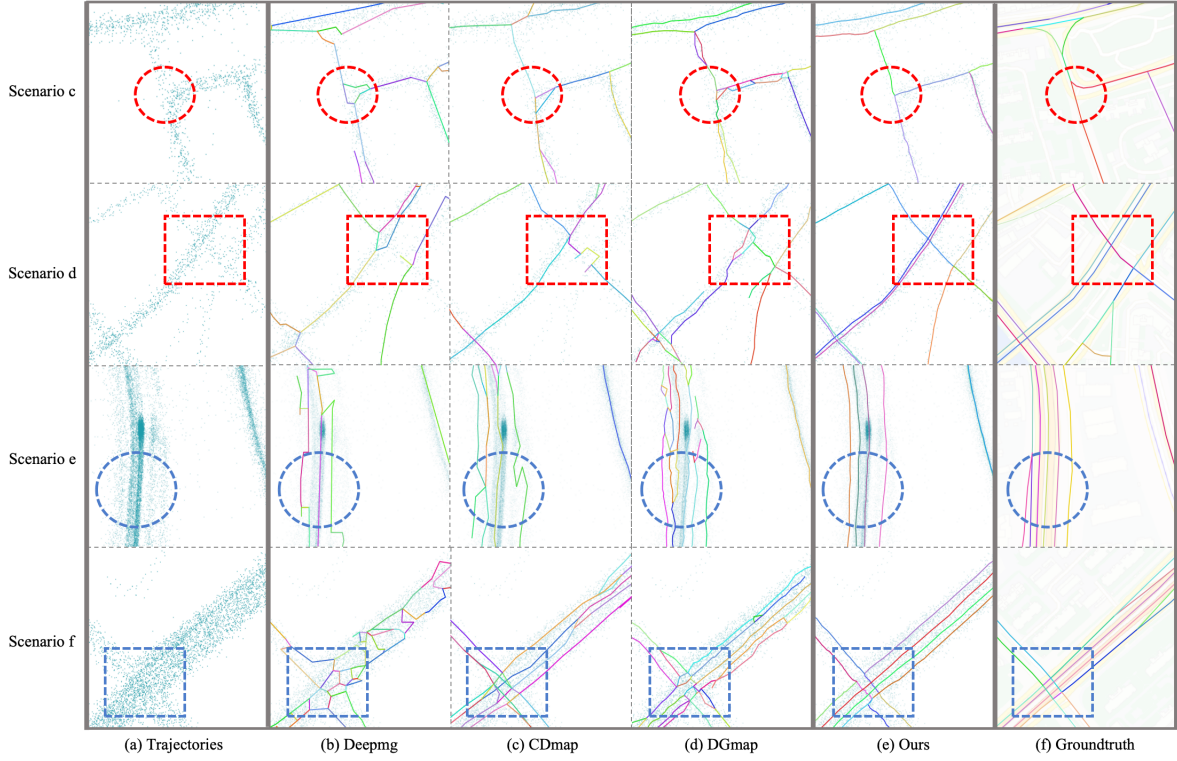


Figure 8: Visualization of inferred road maps

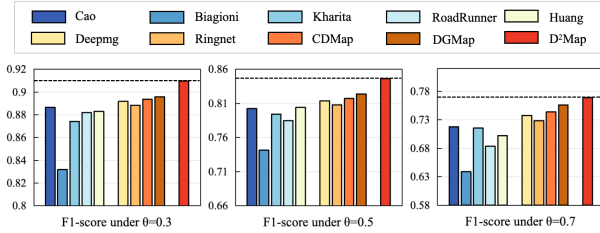


Figure 9: F1-score on SH18 for Map Updating

B Extra Experiments

Extra Visualization Performance. As shown in Figure 8, we further present four cases from two datasets. *Scenario c* demonstrates D²Map’s capability in accurately generating standard road topologies, i.e., T-shape road intersections and crossroads. Although existing methods achieve acceptable performance in identifying them, they still inevitably produce some errors, e.g., Deepmg and CDmap generate redundant shortcuts at road intersections (marked by red circles). In contrast, D²Map generates more reasonable results. *Scenario d* shows another example of the overpass. The existing SOTA methods all mistakenly identify overpass roads that are not connected as crossroads (marked by red rectangles). In contrast, D²Map successfully recognizes the correct topology of overpass roads. *Scenario e* presents a typical parallel road topology. CDMap and DGMap produce both fragmented roads and spurious shortcuts

(marked by blue circles), and D²Map effectively avoids these errors. *Scenario f* shows a complex road topology around a road intersection, combining both parallel roads and overpass structures. In this challenging setting, D²Map accurately distinguishes parallel roads and correctly captures both ground-level intersections and overpass connections (marked by blue rectangles). In contrast, existing methods fail to deliver satisfactory results: they not only generate spurious shortcuts, but also misidentify the connectivity between overpass roads.

Impact on Map Updating. The map update result on SH18 dataset is shown in Figure 9.

Efficiency Analysis. Due to space limitations, we report only key efficiency metrics. The number of parameters and inference time per sample on WX20 are summarized as follows: D²Map (70.4M parameters, 0.110 s/sample) achieves competitive efficiency without any post-processing refinement, compared to DeepMG [27] (63.0M parameters, 0.161 s/sample) and DGMap [30] (33.4M parameters, 0.119 s/sample), both of which require additional refinement steps.

Comparison with Satellite-based Map Inference Methods. Although existing methods for satellite-based map inference are designed for a different input modality, they can be adapted to our task as a baseline. Specifically, we feed the grid feature matrices of the tiles into *Sat2graph*, *TDRoad* and *Road2graph* to get their inferred road maps. The evaluation process is the same as the quantitative evaluation. As shown in Table 4, D²Map outperforms all three methods in F1-score on the two datasets, which further demonstrates its robustness.

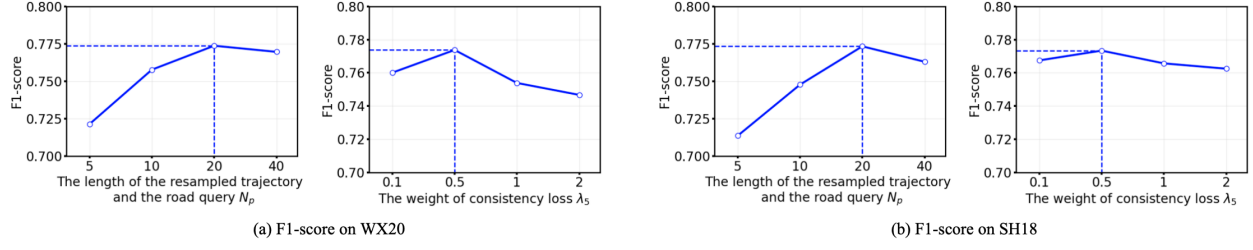


Figure 10: Parameter Sensitivity Analysis

Table 4: Performance Evaluation with methods in the field of map inference from satellite images

Dataset	Methods	Evaluation Criteria			
		TOPO($d_t=10m$)			APLS
		Precision	Recall	F1-score	
WX20	Sat2graph20	0.7318	0.6005	0.6597	0.5642
	TDRoad22	0.7534	0.5615	0.6434	0.5384
	Road2graph24	0.7941	0.5532	0.6521	0.5737
	D ² Map	0.8509	0.7094	0.7738	0.6771
SH18	Sat2graph20	0.7206	0.5961	0.6524	0.5429
	TDRoad22	0.7159	0.5701	0.6347	0.5208
	Road2graph24	0.7635	0.5840	0.6618	0.5547
	D ² Map	0.8672	0.6980	0.7734	0.6650

Parameter Sensitivity. We investigate the impacts of the following parameters on performance: i) The length N_p for both resampled trajectories and road queries N_q ; ii) The weight of the consistency loss λ_5 . We evaluate the F1-score of D²Map on two datasets by varying $N_p = 5, 10, 20, 40$ and $\lambda_5 = 0.1, 0.5, 1, 2$. The results are reported in Figure 10.

We observe that D²Map achieves optimal performance when $N_p = 20$ and $\lambda_5 = 0.5$ on both datasets. While N_p is too small, the polyline resolution is insufficient to accurately model road morphology and fine geometric details. In contrast, the increased sequence length raises the optimization complexity and leads to performance degradation. Regarding λ_5 , the optimal value is found to be 0.5. A too large number of λ_5 means D²Map pays too much attention to maintain the consistency of two graphs, which decreases the performance. Conversely, when λ_5 is set too small, the dual-map consistency is insufficiently enforced, resulting in misaligned predictions.

Visualization of Expert Weights. The experts in fusion strategy-adaptive fusion module are designed to execute different fusion strategies. To validate the effectiveness of the expert selection mechanism, we choose a representative tile containing overpasses and visualize the expert weights assigned to each map element query within this region. As shown in Figure 11, each road is represented as a colored polyline and we select two roads whose 2D projections intersect but are not actually connected, termed as Example road 1 and Example road 2. We then visualize the weights of the router $\text{Softmax}(MLP_3(\mathbf{q}_g || \mathbf{q}_t))$ in the final layer of the transformer decoder for these two roads, where colors closer to red indicate a

higher probability of selecting Expert 1 (only adopt the rasterization view), green indicates Expert 2 (only adopt the serialization view) and blue indicates Expert 3 (fuse two views). It is observed that the point queries located at the intersection of the two roads both select Expert 2 (highlighted by orange circles), i.e., select only serialized view in embedding map elements, while the remaining queries are mainly routed to Experts 1 and 3. This pattern aligns with the design principle: the serialized trajectory representation, which is emphasized in Expert 2, is particularly effective for capturing road adjacency relationship in multi-level structures.

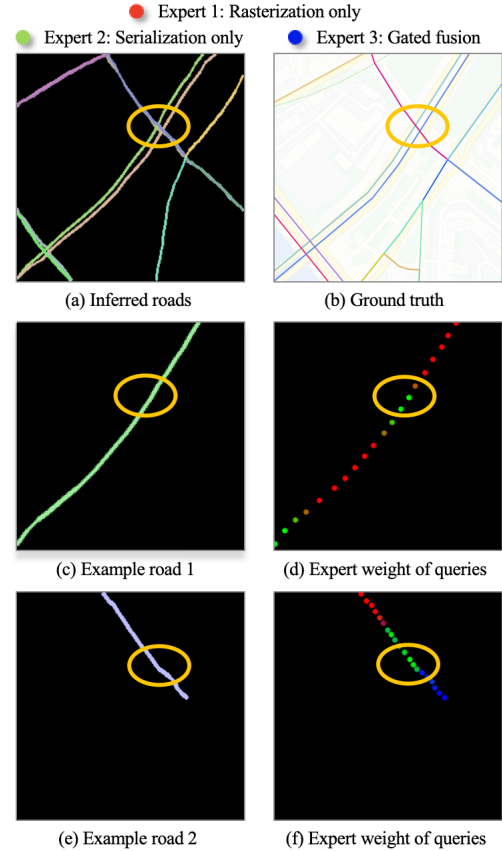


Figure 11: Visualization of expert weights